

Spike: Task 11**Title:** Game Graphs from Data**Author:** Mohamed Shafi Uzman Fassy, 102608927**Goals / deliverables:**

The goal of this spike task is to demonstrate my understanding of data structures, reading and outputting from files that I have learnt from previous tasks in order to create a game world as a graph data structure.

Besides this report, what else was created?

- Code files in see COS30031-2023-102608927\11 - Spike - Game Graphs from Data\GameWorld\GameWorld

Technologies, Tools, and Resources used:

List of information needed by someone trying to reproduce this work.

- Visual Studio 2022 Community Edition
- GitHub
- Game Specification Details Canvas Module
- OO Concepts, Patterns & Structures Canvas Modules
 - 2.1 Software Architecture of Games Lecture Slides
 - 2.2 Game States and Stages Lecture Slides
 - 3.2 Data Structures
 - 3.3 Design Patterns

Tasks undertaken:

- Evaluating Different Data Structures for Inventory Systems
- Download and install Visual Studio
- Download and install Visual Studio C++ Debugging Packages
- Create and Configure VS Project File with a .cpp file to build.
- Importing **nlohmann::json** library.
- Creating Location

File Format Design

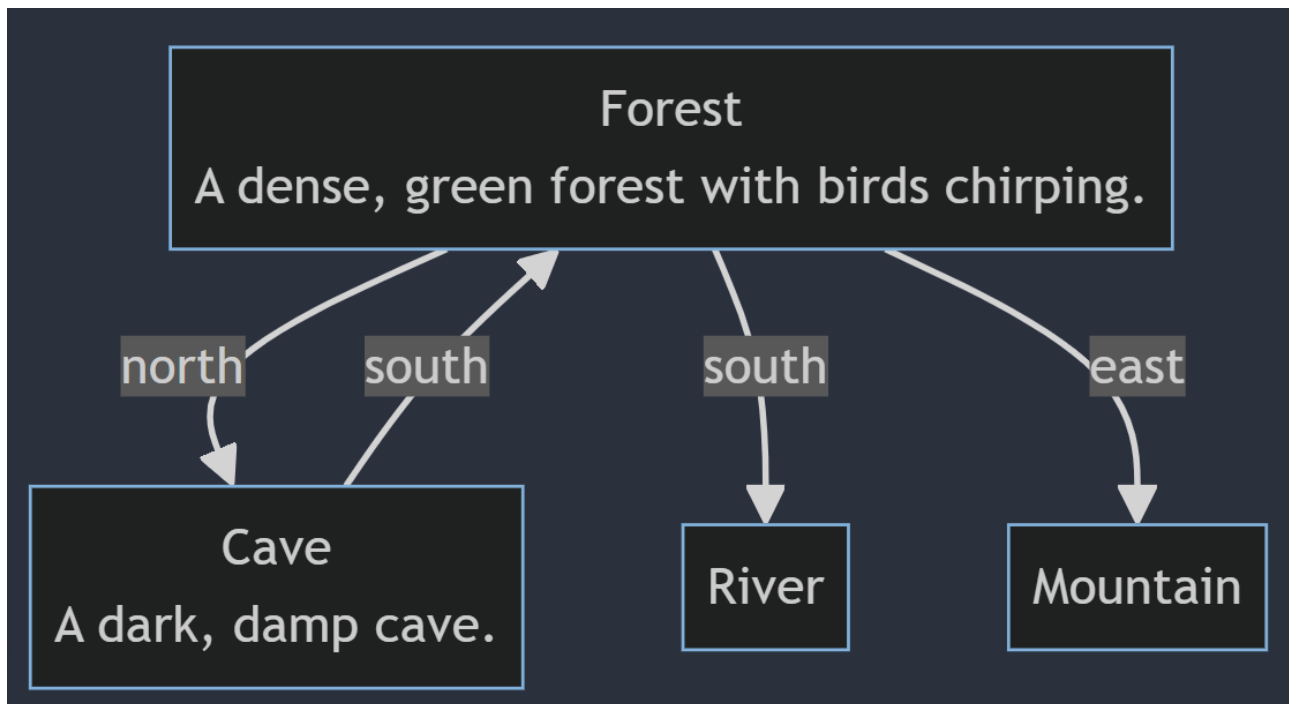


Figure 1 Text File Format Design

To create a game world adventure that loosely takes inspiration from Zorkish Adventure Game I have a list of locations and their connections where each location has a unique location identifier, a location description, and a list of possible connections / directions to that can be taken from the location.

```
1 {
2   "forest": {
3     "description": "A dense, green forest with birds chirping.",
4     "connections": {
5       "north": "cave",
6       "south": "river",
7       "east": "mountain"
8     }
9   },
10  "cave": {
11    "description": "A dark, damp cave.",
12    "connections": {
13      "south": "forest"
14    }
15  }
16 }
```

We created a .JSON file where the root object is locations which will basically contain locations in an array and each location object holds name, description and connections for each location.

Implement the Classes

- Implement Location Class:

```
1  #pragma once
2  #include <unordered_map>
3  #include <string>
4
5  class Location {
6  public:
7      std::string name;
8      std::string description;
9      std::unordered_map<std::string, Location*> connections;
10
11     Location(const std::string& name, const std::string& description);
12 };
```

```
1  #include "Location.h"
2
3  Location::Location(const std::string& name, const std::string& description)
4      : name(name), description(description) {}
```

The **Location.h** file defines the class named **Location** where it represents a specific place within the game world and has a constructor that allows initialization of **Location** object using its name and description. It contains three public member variables:

1. **name** – Represents the name of the location (e.g., "forest" or "cave").
2. **description** – Provides a textual description of the location to give context to the player (e.g., "A dense, green forest with birds chirping.").
3. **connections** – A map (specifically an **unordered_map**) that associates directions (e.g., "north", "south") with pointers to other **Location** objects, indicating where the player can travel from the current location.

The **Location.cpp** file provides a implementation for the **Location** class. Specifically, it contains the definition for the constructor used in **Location** class to initializes the **name** and **description** of the **Location** object based on the provided arguments.

- Testing Locations:

```

1  #include <iostream>
2  #include <fstream>
3  #include <unordered_map>
4  #include "json.hpp"
5  #include "location.h"
6
7  using json = nlohmann::json;
8
9  std::unordered_map<std::string, Location*> world;
10
11 void loadGameWorld(const std::string& filename);
12 void playGame();
13
14 int main() {
15     loadGameWorld("world.json");
16     playGame();
17
18     for (const auto& pair : world) {
19         delete pair.second;
20     }
21
22     return 0;
23 }
24
25 void loadGameWorld(const std::string& filename) {
26     std::ifstream file(filename);
27     if (!file) {
28         std::cerr << "Error opening file: " << filename << std::endl;
29         exit(EXIT_FAILURE);
30     }
31
32     json gameData;
33     file >> gameData;
34
35     for (const auto& [name, locationData] : gameData.items()) {
36         std::string description = locationData["description"];
37
38         Location* loc = new Location(name, description);
39         world[name] = loc;
40     }
41
42     for (const auto& [name, locationData] : gameData.items()) {
43         auto& connections = locationData["connections"];
44
45         for (const auto& [direction, locName] : connections.items()) {
46             world[name]->connections[direction] = world[locName];
47         }
48     }
49 }
50
51 void playGame() {
52     Location* currentLocation = world.begin()->second;
53
54     while (true) {
55         std::cout << "You are at: " << currentLocation->name << std::endl;
56         std::cout << currentLocation->description << std::endl;
57         std::cout << "Available directions: ";
58         for (const auto& [direction, _] : currentLocation->connections) {
59             std::cout << direction << " ";
60         }
61         std::cout << "\nChoose a direction (or type 'quit' to exit): ";
62         std::string choice;
63         std::getline(std::cin, choice);
64
65         if (choice == "quit") {
66             std::cout << "Goodbye!\n";
67             break;
68         }
69
70         if (currentLocation->connections.find(choice) != currentLocation->connections.end()) {
71             currentLocation = currentLocation->connections[choice];
72         }
73         else {
74             std::cout << "Invalid direction!\n";
75             if (std::cin.fail()) {
76                 std::cin.clear();
77                 std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
78             }
79         }
80     }
81 }
82 }
83

```

```

You are at: forest
A dense, green forest with birds chirping.
Available directions: north east south
Choose a direction (or type 'quit' to exit): north
You are at: cave
A dark, damp cave.
Available directions: south
Choose a direction (or type 'quit' to exit): nuth
Invalid direction!
You are at: cave
A dark, damp cave.
Available directions: south
Choose a direction (or type 'quit' to exit):

```

Based on the output we can see that the program runs and is able to read the .JSON file that contains the locations and the output displays where the player is located current, gives a description of location and provides the possible connections/directions in that location and handles for user input to move between locations.

In main class we load the JSON file using the **loadGameWorld()** function by attempting to open the file and can handle errors if file does not exist where it exits the program. It reads the data and populates the world map with **Location** objects and sets up the connection between using the connections data in the JSON file.

The **playGame()** function implements the gameplay loop where it starts of the player in the first location in the world map as the current location and displays description of location and available directions / connections the player can travel in. It handles player input as well for moving to directions or exits the game if player types in "quit". It handles input errors as well by displaying error messages such as "invalid direction" since the required input is only direction based.